GU Tech - Al Ghazali University

Midterm Exam Fall 2025

Department of Computer Science

Object Oriented Programming (Lab)

20 November 2025 10:00 am - 12:30 pm Total marks: 45

Instructions

- Attempt all questions. There are 3 questions for a total of 45 marks.
- This is an open book exam but peer discussions and Internet access are not allowed. You may bring and use your laptops and lab manuals.
- You may use any indentation style but be consistent.
- Write clean code. Dirty hacks are grounds for marks deduction. They include but are not limited to premature `return` statements in `void` functions, unnecessarily breaking out of `for` loops, etc.
- Do **not** submit .exe files. Submit only the code files.

Questions

1. Create a class hierarchy for vehicles: (15)

   - **Base Class:** `Vehicle`
     - Data members: `brand (string)`, `year (int)`.
     - Default constructor initializing brand to `"Generic"`, year to `2020`.
     - Parameterized constructor taking brand and year.
     - Member function `displayInfo()` that shows vehicle details.
   - **Derived Class:** `Car` (inherits from `Vehicle`)
     - Additional data members: `doors (int)`, `fuelType (string)`.
     - Constructor using member initializer list that chains to Vehicle's parameterized constructor.
     - Override `displayInfo()` to show car-specific details along with vehicle information.
   - **Derived Class:** `Motorcycle` (inherits from `Vehicle`)
     - Additional data members: `engineCapacity (int)`, `hasFairing (bool)`.
     - Constructor using member initializer list with proper constructor chaining.
     - Override `displayInfo()` method.

   Test your classes by:

   - Creating `Car` and `Motorcycle` objects using constructors with proper chaining.
   - Demonstrating function overriding by calling `displayInfo()` on base class pointers pointing to derived objects.

2. Create a banking system with proper memory management: (15)

- **Base Class:** `BankAccount`
  - Data members: `char* accountNumber`, `char* accountHolderName`, `double balance`.
  - Virtual functions: `deposit(double amount)`, `withdraw(double amount)`, `displayAccountInfo()`.
- **Derived Class:** `SavingsAccount`
  - Additional members: `double interestRate`, `double minimumBalance`.
  - Override `withdraw()` to enforce minimum balance.
  - Function: `applyInterest()`.
- **Derived Class:** `CheckingAccount`.
  - Additional members: `double overdraftLimit`.
  - Override `withdraw()` to allow overdraft up to limit.
    - **Overdraft:** A deficit in a bank account caused by drawing more money than the account holds.

Demonstrate polymorphism and make sure to avoid shallow copy issues and ensure that the following code snippet works correctly:

```
CheckingAccount a(accountNumber, accountHolderName,
interestRate, minimumBalance);

CheckingAccount b = a;

char someName[] = "Spelled Correctly";
b.setAccountHolderName(someName);

a = b;

cout << a.getAccountHolderName() << "\n";
```

3. Create a Fraction class that can handle basic arithmetic operations. (15)

```
class Fraction {
private:
    int numerator, denominator;

    // Helper function to find GCD
    int gcd(int a, int b) {
        if (b == 0) return a;
        return gcd(b, a % b);
    }

    // Helper function to simplify fraction
    void simplify() {
        int g = gcd(abs(numerator), abs(denominator));
        numerator /= g;
        denominator /= g;
        if (denominator < 0) {
            numerator = -numerator;
            denominator = -denominator;
        }
    }
public:
    // Constructor
    Fraction(int num = 0, int den = 1)
    : numerator(num), denominator(den) {
        if (denominator == 0) {
            cout << "Error: Denominator cannot be zero!" << endl;
            denominator = 1;
        }
        simplify();
    }

    // TODO: Implement these operators
    // Fraction operator + (const Fraction& other) const;
    // Fraction operator - (const Fraction& other) const;
    // Fraction operator * (const Fraction& other) const;
    // Fraction operator / (const Fraction& other) const;
    // bool operator == (const Fraction& other) const;
};
```